

DIGITAL LIBRARY MANAGEMENT SYSTEM USING TREE HIERARCHY

1. Abstract

A Digital Library Management System is a software application designed to manage and organize digital resources such as books, journals, and documents. It provides users with easy access, search functionality, and efficient data storage. By using a **tree hierarchy structure**, the system organizes data in a parent-child relationship, making retrieval faster and structured. This system reduces manual work and improves accuracy, efficiency, and accessibility.

2. Introduction

A Digital Library replaces traditional libraries by storing information digitally. Users can access materials anytime and anywhere. The system manages large amounts of data efficiently.

The **tree data structure** is used to represent categories and subcategories of books. For example:

- Root → Library
- Child → Genres (Science, Literature)
- Sub-child → Books

This hierarchical structure improves searching and data organization.

3. Objectives

- To store and manage digital books efficiently
- To provide fast search using hierarchical structure
- To reduce manual effort in library management
- To organize books using tree-based classification
- To improve accessibility and data retrieval

4. Scope of the Project

The scope of the Digital Library Management System using Tree Hierarchy includes the following:

- The system is designed to **store, manage, and organize digital books** in a structured way.
- It uses a **tree hierarchy structure** to classify books into categories and subcategories for easy access.
- Users can **search, view, and retrieve books quickly** based on categories.

5. Limitations

The Digital Library Management System using Tree Hierarchy has the following limitations:

- The system is suitable mainly for **small to medium-sized libraries**; handling very large data may reduce performance.
- Tree structure can become **complex and difficult to manage** when the number of categories and subcategories increases.
- Searching is limited to the **hierarchical path**, so advanced search features may not be fully supported.

6. Literature Review

Digital Library Management Systems have evolved from simple manual systems to advanced digital platforms for efficient data storage and retrieval. Various data structures have been studied and used to improve performance and organization. Among them, **Arrays, Linked Lists, and Trees** play an important role.

Arrays in Digital Library

Arrays are one of the simplest data structures used in early systems. They store data in a **sequential manner**. Researchers found that arrays are easy to implement and provide **fast access using index values**. However, arrays have limitations such as **fixed size** and difficulty in insertion and deletion, making them less suitable for dynamic digital libraries.

Linked List in Digital Library

Linked Lists were introduced to overcome the limitations of arrays. In this structure, elements are stored in **nodes connected by pointers**. Studies show that linked lists allow **dynamic memory allocation** and easy insertion or deletion of records. However, searching in linked lists is **time-consuming**, as it requires sequential traversal.

Tree in Digital Library

Tree data structures are widely used in modern digital library systems. A tree organizes data in a **hierarchical structure**, where each node represents a category or book. Researchers highlight that trees provide **efficient searching, better organization, and scalability**. The parent-child relationship helps in categorizing books into genres and subcategories, making navigation easier.

7. Methodology

The Digital Library Management System is developed using a structured approach to organize and manage digital resources efficiently.

Data Structures Used

The following data structures are used in the system:

1. Tree

- Main data structure used for organizing data
- Represents **hierarchical relationships** (Library → Category → Books)
- Supports efficient searching and classification

2. Linked List (Optional Use)

- Used to manage multiple child nodes at the same level
- Helps in dynamic memory allocation

3. Arrays (Basic Use)

- Used for temporary storage or input handling
- Simple and easy to implement

Functions Used

The system includes the following functions:

- **createNode()**
Creates a new node with book or category name
- **addChild()**
Adds a child node to a parent node
- **display()**
Displays the tree structure in hierarchical form
- **search()** (*optional*)
Searches for a specific book or category
- **deleteNode()** (*optional*)
Removes a node from the tree

Algorithms

1. Insertion Algorithm (Add Node)

1. Create a new node

2. Check if the parent node exists
3. If no child exists, attach directly
4. Otherwise, traverse siblings
5. Add the new node at the end

2. Traversal Algorithm (Display Tree)

1. Start from root node
2. Print current node
3. Recursively traverse child nodes
4. Move to sibling nodes
5. Repeat until all nodes are displayed

3. Searching Algorithm

1. Start from root
2. Compare node value with target
3. If matched, return result
4. Otherwise, search child nodes
5. Continue until found or end

8. Technology Used

The Digital Library Management System using Tree Hierarchy is developed using the following technologies:

1. Programming Language

- **C Language**
Used to implement the system logic, data structures (tree), and functions.

It provides **efficient memory management** and fast execution.

2. Data Structures

- **Tree Data Structure**
Used to organize books in a hierarchical manner (Library → Category → Books).
- **Linked List (Optional)**
Used for managing child nodes dynamically.
- **Arrays**
Used for temporary storage and input handling.

3. Development Tools

- **Code Editor (e.g., Turbo C / VS Code)**
Used for writing and editing the program code.
- **Compiler (GCC)**
Used to compile and execute the C program.

4. Operating System

- **Windows / Linux**
Platform used to develop and run the application.

5. Concepts Used

- **Dynamic Memory Allocation**
(malloc, free)
- **Pointers and Structures**
- **Tree Traversal Techniques**
- **Modular Programming (Functions)**

6. User Interface

- **Console-Based Interface**
Simple text-based interaction for input and output.

9. System Architecture

The Digital Library Management System follows a **simple layered architecture** to manage data efficiently.

Architecture Overview

The system consists of three main layers:

1. Presentation Layer (User Interface)

- Provides interaction between user and system
- Accepts user input (add, search, display books)
- Displays output results

- Implemented using a **console-based interface**

2. Application Layer (Logic Layer)

- Contains the main program logic
- Processes user requests
- Performs operations like **insertion, searching, deletion, and traversal**
- Controls the flow of the program

3. Data Layer (Storage Layer)

- Stores data using **tree data structure**
- Maintains hierarchical relationships (parent-child nodes)
- Handles memory allocation and data organization

Architecture Flow

User → Input → Application Logic → Tree Data Structure → Output Display

10. System Designing

System design defines how the system components are organized and work together.

1. Input Design

- User enters:
 - Book name
 - Category / Subcategory
- Input is validated before processing

2. Output Design

- Displays:
 - Tree structure of library
 - Search results
 - Messages (success/error)
- Output is shown in **clear hierarchical format**

3. Data Design

- Data is stored in **nodes of tree structure**
- Each node contains:
 - Name (book/category)
 - Pointer to child
 - Pointer to sibling

4. Process Design

- **Add Book** → Insert node into tree
- **Search Book** → Traverse tree
- **Display Library** → Use recursive traversal
- **Delete Book** → Remove node and adjust links

5. Module Design

The system is divided into modules:

- **Node Creation Module** → createNode()
- **Insertion Module** → addChild()
- **Traversal Module** → display()
- **Search Module** → search()
- **Deletion Module** → deleteNode()

11. Implementation Details

The Digital Library Management System is implemented using the **C programming language** and follows a structured approach using tree hierarchy.

1. Program Structure

- The program is divided into multiple functions for modularity
- Each function performs a specific task like creation, insertion, and display
- The system starts execution from the **main() function**

2. Node Implementation

- Each element in the library is represented as a **node**
- A node contains:
 - **Name** (book/category)
 - **Pointer to child** (subcategories/books)

- **Pointer to sibling** (same level nodes)
- Memory is allocated dynamically using **malloc()**

3. Tree Construction

- The root node represents the **main library**
- Categories are added as **child nodes**
- Subcategories and books are added under respective parent nodes
- Sibling pointers connect nodes at the same level

4. Functional Implementation

a) Node Creation

- A new node is created using `createNode()`
- Memory is allocated and values are initialized

b) Insertion

- New nodes are inserted using `addChild()`
- If the parent has no child, node is added directly
- Otherwise, it is added at the end of sibling list

c) Traversal (Display)

- Implemented using **recursion**
- Displays the hierarchy level by level
- Indentation is used to represent levels

d) Searching (Optional)

- Traverses the tree recursively
- Compares each node with the target value

e) Deletion (Optional)

- Removes a node from the tree
- Adjusts pointers to maintain structure

5. Execution Flow

1. Start the program
2. Create root node (Library)
3. Add categories and books
4. Perform operations (insert, search, display)
5. Display final tree structure

6. Memory Management

- Dynamic memory allocation is used
- Efficient use of pointers ensures flexibility
- Memory can be freed using `free()` (if implemented)

7. Output Representation

- The output is displayed in a **hierarchical format**
- Indentation shows levels of tree
- Example:

```
Library
  Science
    Physics
    Chemistry
  Literature
    Poetry
    Novels
```

12. Result and Output

Result

The Digital Library Management System using Tree Hierarchy was successfully implemented using the C programming language. The system is able to:

- Organize books in a **hierarchical structure**
- Perform operations like **insertion, searching, and display**
- Provide **efficient data retrieval** using tree traversal
- Reduce manual effort and improve data management

The use of tree structure ensures **better organization and faster access** compared to linear data structures.

Output

The output of the system is displayed in a **hierarchical format**, representing the structure of the digital library.

Sample Output

Digital Library Tree Structure:

```
Library
  Science
    Physics
    Chemistry
  Literature
    Poetry
    Novels
```

13. Conclusion

The Digital Library Management System using Tree Hierarchy is an efficient solution for organizing and managing digital resources. The system successfully demonstrates how a **tree data structure** can be used to represent hierarchical relationships between categories, subcategories, and books.

By implementing this system, we achieve:

- **Efficient data organization** using parent-child relationships
- **Faster searching and retrieval** of information
- **Reduced manual effort** in managing library records
- **Scalability** to add more categories and data

The use of **modular functions, pointers, and dynamic memory allocation** makes the system flexible and easy to maintain. Although the system has some limitations, it provides a strong foundation for developing more advanced digital library applications.

14. Future Work

The current Digital Library Management System can be further improved with the following enhancements:

- Develop a **Graphical User Interface (GUI)** to make the system more user-friendly instead of a console-based interface.
- Implement a **database system (MySQL/SQLite)** for permanent data storage instead of temporary memory.
- Add **user authentication (login/signup)** to ensure security and controlled access.

- Introduce an **advanced search feature** (by author, title, keywords).
- Enable **multi-user access** to allow multiple users to use the system simultaneously.

15. References

1. Ellis Horowitz, Sartaj Sahni, Susan Anderson-Freed,
Fundamentals of Data Structures in C, Universities Press.
2. Mark Allen Weiss,
Data Structures and Algorithm Analysis in C, Pearson Education.
3. Reema Thareja,
Data Structures Using C, Oxford University Press.
4. Seymour Lipschutz,
Data Structures (Schaum's Outline Series), McGraw Hill.
5. Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman,
Data Structures and Algorithms, Pearson.

